UNIT 1 FOUNDATIONS OF OOP AND C++ ESSENTIALS

History and evolution of C++ – Difference between C and C++ – Structure of a C++ program – Namespaces – References vs Pointers – OOP Principles: Encapsulation, Abstraction, Inheritance, Polymorphism – Classes and Objects – Access Specifiers – Constructors and Destructors – Constructor overloading – Copy Constructor – this pointer – static members – const members – Friend functions and Friend classes – Scope resolution operator.

History and Evolution of C++

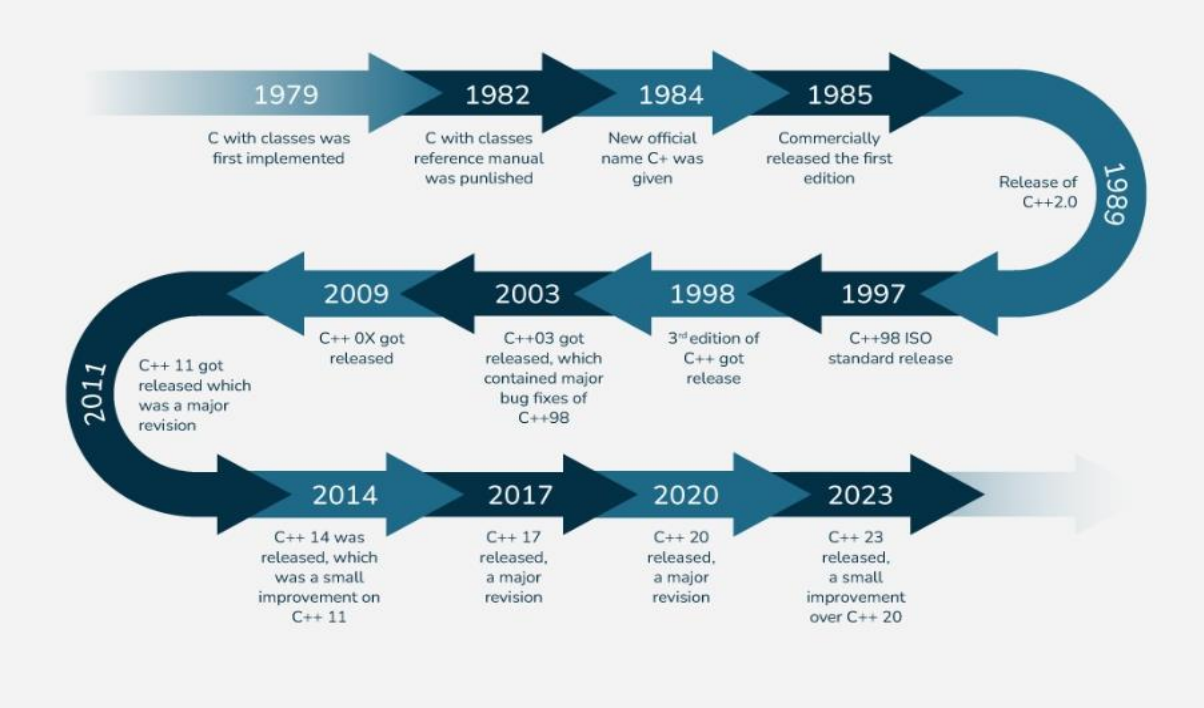
C++ is a **general-purpose, middle-level, compiled programming language** that supports multiple programming paradigms. It was designed to provide the efficiency and flexibility of the C programming language while introducing powerful features for developing large and complex software systems.

C++ supports:

- Procedural programming
- Object-oriented programming
- Generic programming
- Functional programming techniques
- Low-level system programming
- High-performance computing

C++ is often described as a **multi-paradigm programming language** because programmers can choose different programming approaches depending on the problem.

The development of C++ was mainly motivated by the need to manage increasingly large software projects more effectively while maintaining the execution efficiency of C.



Important milestones

1979 – C with Classes

Bjarne Stroustrup began developing an extension of C called **C with Classes**.

It introduced concepts such as:

- Classes
- Objects
- Encapsulation
- Inheritance
- Virtual functions

1983 – C++

The name **C++** was introduced.

The ++ operator in C means increment, symbolically suggesting an enhanced version of C.

1998 – C++98

The first standardized version of C++.

Important features included:

- Object-oriented programming
- Templates
- Standard Template Library
- Exceptions
- Namespaces
- Standard library

2003 – C++03

A refinement of C++98 with corrections and improvements.

2011 – C++11

A major modernization of C++.

Important features:

- auto
- Range-based for
- Lambda expressions
- Smart pointers
- Move semantics
- nullptr
- enum class
- constexpr

2014 – C++14

Improved features introduced in C++11, especially generic programming and constexpr.

2017 – C++17

Important features:

- Structured bindings
- if constexpr
- std::optional

- std::variant
- std::string_view
- Filesystem library

2020 – C++20

Important features:

- Concepts
- Ranges
- Coroutines
- Modules
- Three-way comparison operator <=>

2023 – C++23

Continued improvements to:

- Ranges
- Standard library
- Modules
- Generic programming
- Modern syntax

The evolution of C++ was driven by the need for:

1. Better software organization -Classes and objects help organize large programs.
2. Code reusability -Inheritance, templates, and libraries allow existing code to be reused.
3. Type safety -Modern C++ provides stronger compile-time checking.
4. Memory safety -Smart pointers and RAII help manage resources safely.
5. Performance-C++ continues to provide high-performance programming capabilities.
6. Generic programming -Templates allow algorithms and data structures to work with different types.
7. Modern programming techniques-Features such as lambdas, ranges, concepts, and co-routines support modern software development.

Difference Between C and C++

C and C++ are popular programming languages used for system programming, application development, game development, and embedded systems. C is a procedural programming language, while C++ is an extension of C that supports both procedural and object-oriented programming concepts.

C focuses mainly on functions and procedures.
C++ supports classes, objects, inheritance, and polymorphism.
Both languages are fast, efficient, and widely used in software development.

Parameter	C	C++
Programming Paradigm	C is a structural or procedural programming language.	C is a structural as well as an object-oriented programming language.
History	C was developed by scientist Dennis Ritchie in 1972 at Bell Laboratories.	C was developed by Bjarne Stroustup in 1979.
Approach	C follows a top-down approach	C follows the bottom-up approach.
Keywords	C contains 32 keywords	C++ contains 63 keywords.
Data Types	C supports built-in data types.	C++ support both built-in and user-defined data types.
File Extension	.c is the file extension for C programming language	.cpp is the file extension for C++ programming language
Header File	<stdio.h> header file is used by C language	<iostream.h> header file is used by C++ language
Allocation and Deallocation of Memory	In C language, we use calloc() and malloc() for dynamic allocation of memory and free() for deallocation of memory.	In C++ language, we use a new operator for the allocation of memory and a delete operator for the deallocation of memory.
Access Modifier	C language does not support access modifier	C++ support access modifier
Reference Variable	C does not support reference variable	C++ support reference variable
Function Overloading and Function Overriding	C don't supports function overloading and function overriding	C++ supports function overloading and function overriding
Exception Handling	C does not support exception handling directly, it uses the function that support exception handling	C++ directly support exception handling with the help of try – catch block

Program Division	C is a procedural language, so code written in C are divided in separate blocks known as function	C++ is a object oriented language, so code and divided into classes and objects
Inline Function	C doesn't support inline function	C++ support inline function
Driven Type	C is known as function driven language	C is known as object driven language
Compatibility	Code written in C language can be run on C++ compiler as C is the foundational language	Code written in C++ language can be run on C compiler as C++ language includes OOP's concept
Data and Function	In C, the data and function are separated as it is a procedural programming language	In C++, the data and function are encapsulated as it is a object oriented programming language
Input and Output Function	In C scanf() and printf() functions are used to take the input and output respectively	In C++ cin and cout functions are used to take the input and output respectively
Application Development	C language is more suitable for low level implementation such as network driver, text editor, assembler, etc	C++ language is more suitable for high level implementation such as game development, smartwatches, embedded systems, etc
Namespace	To prevent the collision and organize the code, namespace is needed but C does not support that	C++ support the namespace

Example	#include <stdio.h>
	int main() {
	printf("Hello, World!\n");
	return 0;
	}
		#include <iostream>
		int main() {
		std::cout << "Hello, World!" << std::endl;
		return 0;
		}

Structure of a C++ Program

A C++ program follows a well-defined structure that organizes different parts of the code, making programs easier to write, understand, and maintain. Each section of a C++

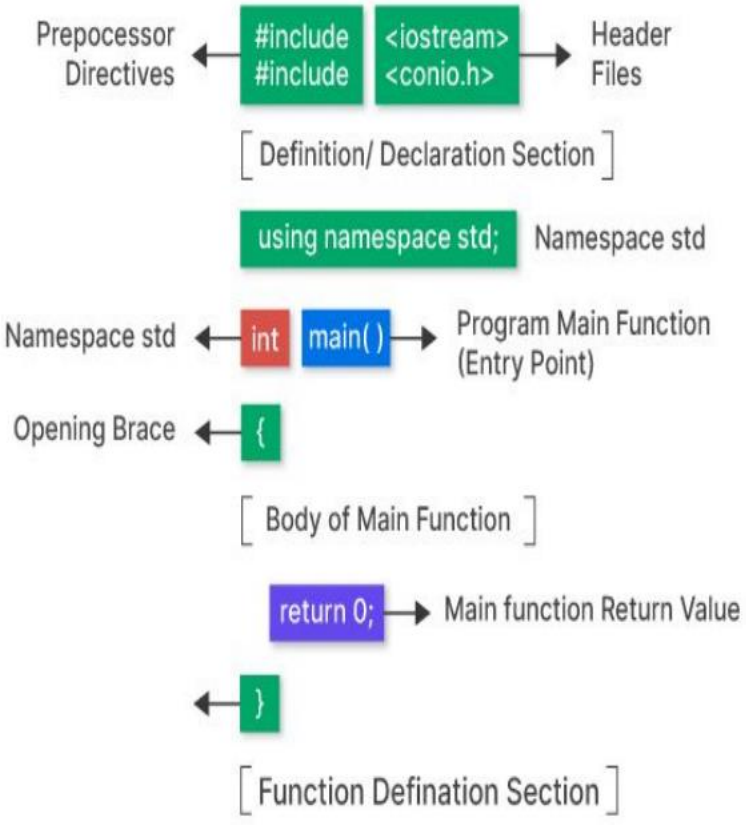
program has a specific purpose, such as including libraries, declaring variables, defining functions, and specifying the starting point of execution.

- Helps organize program components in a logical manner.
- Improves code readability and maintainability.
- Separates declarations, definitions, and executable statements.
- Provides a standard framework followed by most C++ programs.

A typical C++ program consists of one or more source code files (.cpp files), each contributing to the program's logic. These files include various elements such as:

- Comments (for documentation)
- Header files (for including standard libraries or user-defined code)
- Namespaces (especially std, for avoiding naming conflicts)
- Global variables and constants
- Function prototypes
- Class and function definitions
- The main() function (the entry point of the program)
- Input/output statements (e.g., cin, cout)

The diagram below illustrates how these components are organized within a basic C++ program



Documentation Section

The documentation section contains comments that describe the purpose, logic, or functionality of the program. Comments improve code readability and help others understand the program more easily.

- Used to explain the program or specific code segments.
- Ignored by the compiler during compilation.
- Optional, but recommended for better code documentation.

Linking Section

The linking section contains the header files required by the program. These headers provide access to predefined functions, classes, and other library features.

Header Files

Header files are included using the #include preprocessor directive.

- Allow the use of standard library features.
- Provide declarations for functions, classes, and objects.
- Help organize reusable code.

```
#include <iostream>
```

The <iostream> header provides standard input and output facilities such as cin, cout, and cerr.

Namespaces

A namespace permits grouping of various entities like classes, objects, functions, and various C++ tokens, etc. under a single name.

- Any user can create separate namespaces of its own and can use them in any other program.
- In the below snippets, namespace std contains declarations for cout, cin, endl, etc. statements.

```
using namespace std;
```

Namespaces can be accessed in multiple ways:
using namespace std;
using std::cout;
using std::cin;

Definition Section

The definition section is used to define constants, macros, and type aliases that can be used throughout the program.

- Used for symbolic constants and custom type names.
- Anyone can define your own datatype using primitive data types.

- Improves code readability and maintainability.
- Helps avoid repeated values in the program.

Main Function

The main() function is the entry point of every C++ program. Program execution always begins from this function.

- Contains the executable statements of the program.
- Controls the overall program flow.
- Returns control to the operating system when execution complete

```
int main()
{
   cout << "Hello World";
   return 0;
}
```

Explanation

- Execution starts from main().
- Statements inside the function are executed sequentially.
- The return 0; statement indicates successful program termination.

Namespaces

A namespace in C++ is a container used to organize identifiers like variables, functions, and classes. It helps prevent naming conflicts when the same names are used in different parts of a program.

- Groups related code under a unique name.
- Prevents collisions between identifiers having the same name.
- Improves code organization and readability.

Syntax

```
namespace namespace_name
{
   // declarations
}
```

Example

```
#include <iostream>
using namespace std;
namespace College
{
   int students = 100;
```

```
}
int main()
{
   cout << College::students;
   return 0;
}
```

Output

100

The **scope resolution operator:** is used to access the member of a namespace.

References vs Pointers

Pointers in C++

Pointers are variables that store the memory address of another variable. They provide direct access to memory locations and are commonly used for dynamic memory allocation, efficient parameter passing, and implementing complex data structures.

Store the address of a variable or memory location.

- Can be reassigned to point to different variables.
- Can store a nullptr value when not pointing to any valid memory location.
- Require the dereference operator (*) to access the value stored at the address.
- Have their own memory address independent of the variable they point to.

Syntax

```
datatype *var_name;
or
datatype* var_name;
```

- datatype represents the type of data being pointed to
- var_name is the name of the pointer variable

Program

```
#include <iostream>
using namespace std;
int main()
{
   int x = 10;          // variable declared
   int* myptr;         // pointer variable
   // storing address of x in pointer myptr
   myptr = &x;
   cout << "Value of x is: ";
   cout << x << endl;
   // print the address stored in myptr pointer variable
```

```
    cout << "Address stored in myptr is: ";
    cout << myptr << endl;
    // printing value of x using pointer myptr
    cout << "Value of x using *myptr is: ";
    cout << *myptr << endl;
    return 0;
}
```

OUTPUT

```
Value of x is: 10
Address stored in myptr is: 0x7ffd2b32c7f4
Value of x using *myptr is: 10
```

- An integer variable x is initialized with the value 10.
- The pointer variable myptr stores the address of x using the address-of operator (&).
- Printing myptr displays the memory address of x.
- Dereferencing the pointer using *myptr accesses the value stored at that address.
- Since myptr points to x, *myptr prints the value 10.

Reference in C++

A reference is an alias or alternative name for an existing variable. Once a reference is initialized, it refers to the same memory location as the original variable, allowing both names to access the same data.

- Act as an alias for an existing variable
- Must be initialized at the time of declaration
- Cannot be reassigned to refer to another variable
- Do not require dereferencing to access the value
- Share the same memory location as the original variable

Uses of References

References are commonly used for passing arguments to functions efficiently. In C++, arguments can be passed in the following ways:

Call by Value: A copy of the argument is passed to the function

Call by Reference using Pointer: The address of the argument is passed using a pointer

Call by Reference using Reference: The argument is passed using a reference variable

Syntax

```
datatype& ref_name = variable;
```

where:

- datatype represents the type of the referenced variable
- ref_name is the name of the reference variable
- variable is the existing variable being referenced

```
#include <iostream>
using namespace std;
int main(){
   int y = 10; // myref is a reference to y.
   int& myref = y;  // changing value of y to 20
   y = 30;
   cout << "value of y is " << y << endl;
   cout << "value of myref after change in value of y is: "<< myref << '\n';
   return 0;
}
```

Output

```
Value of y is 30
Value of myref after change in value of y is: 30
```

Explanation

- An integer variable y is initialized with the value 10
- myref is declared as a reference to y
- Both y and myref refer to the same memory location
- When the value of y is changed to 30, the value accessed through myref also becomes 30
- Printing either y or myref displays the same value because they refer to the same variable

Reference vs Pointer

Pointers and references are important C++ features used to access and manipulate data efficiently. A pointer stores the memory address of a variable, while a reference acts as an alias for an existing variable.

- Pointers store memory addresses and provide indirect access to data.
- References create an alternative name for an existing variable.
- Both are commonly used for efficient parameter passing and data manipulation.

Feature	Pointer (*)	Reference (&)
Definition	A distinct variable holding a memory address.	An alias for an existing variable.
Initialization	Optional at declaration time.	Mandatory at declaration time.
Reassignment	Can be changed to point to another variable.	Cannot be reassigned once initialized.
Null Value	Can be nullptr.	Cannot be null; must bind to a valid object.
Syntax	Requires explicit dereferencing (*ptr).	Handled implicitly like a standard variable.
Arithmetic	Supports pointer arithmetic (e.g., ptr++).	Does not support arithmetic operations.
Memory Address	Has its own independent address in memory.	Shares the address of the variable it aliases.

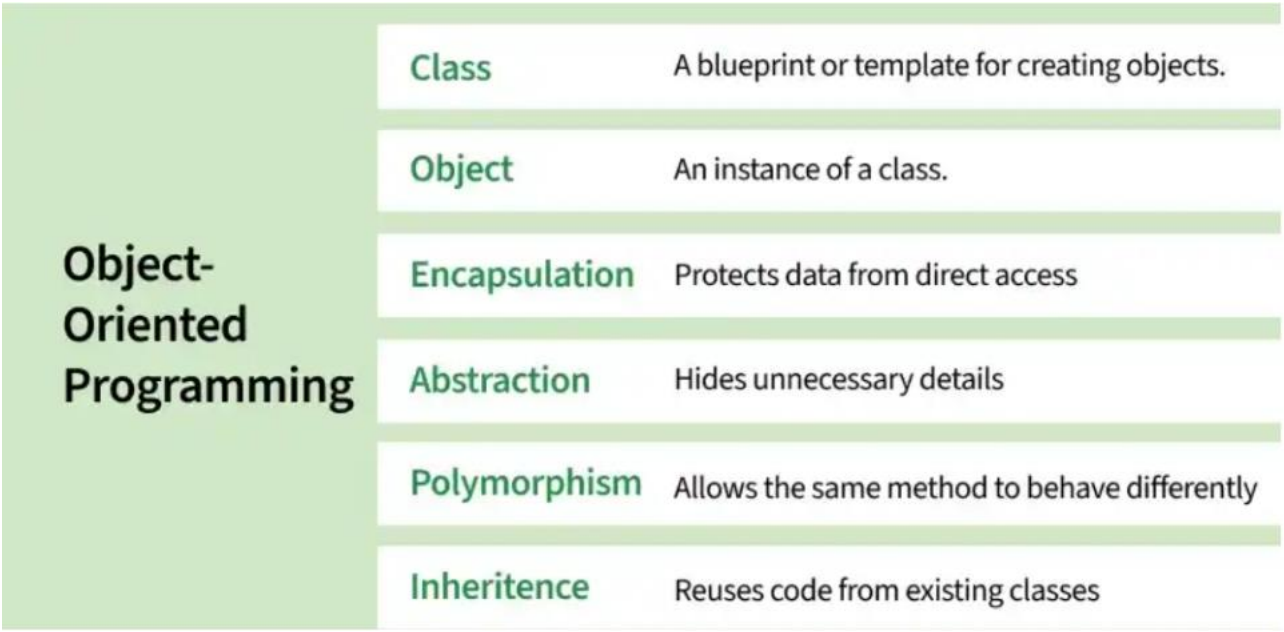
OOP Principles -Object-Oriented Programming Principles

Object-Oriented Programming (OOP) is a programming paradigm that organizes programs around classes and objects. A class defines the data members and member functions, while an object is an instance of a class. OOP helps build modular, reusable, and maintainable software by modeling real-world entities.

- Promotes code reusability through inheritance.
- Improves code maintainability and scalability.
- Provides data security through encapsulation.
- Models real-world entities using classes and objects.

Characteristics of OOP

Object-Oriented Programming is characterized by several fundamental concepts that make software more modular, reusable, and easier to maintain.



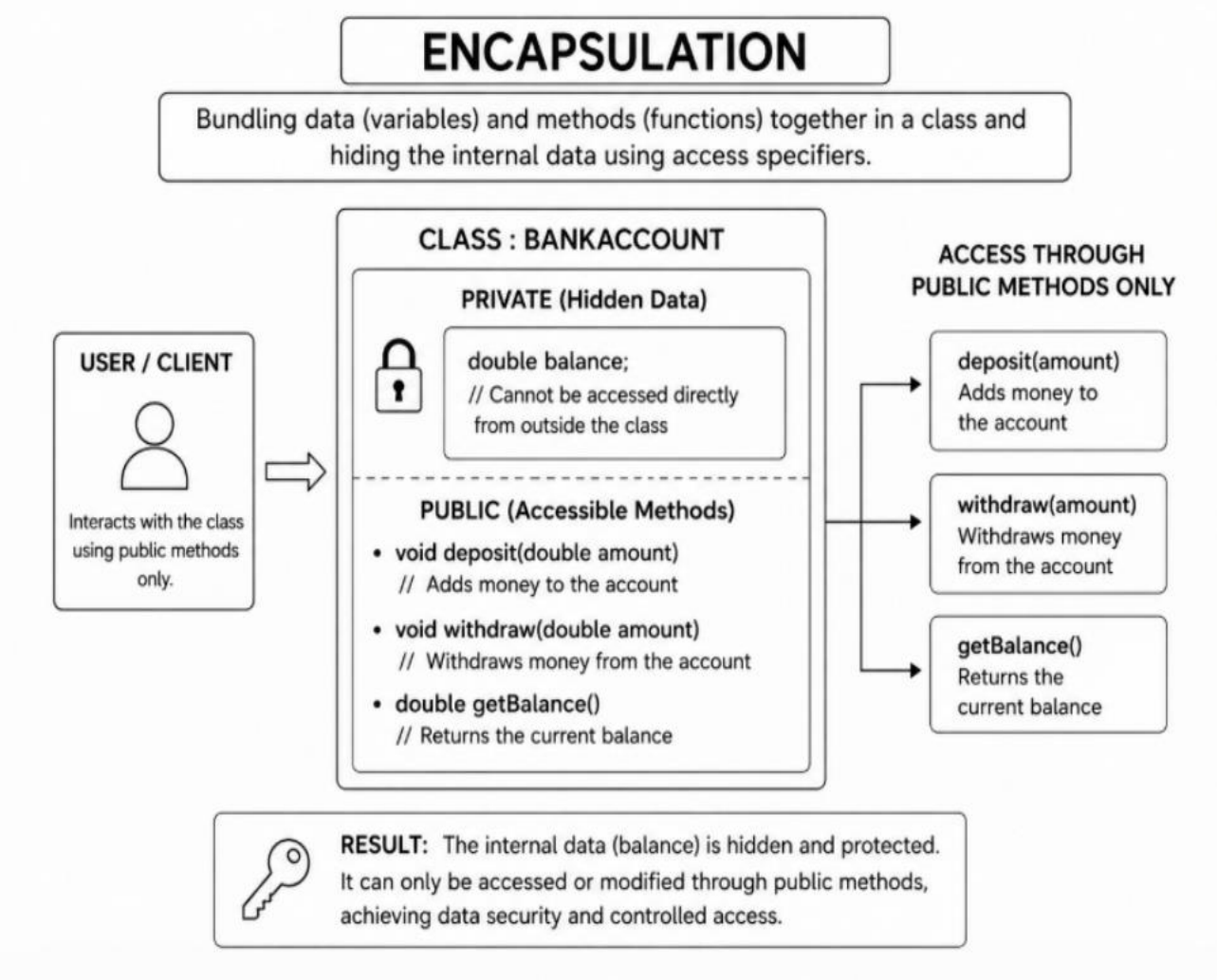
Encapsulation, Abstraction, Inheritance, Polymorphism

Encapsulation

Encapsulation is the process of combining data members and member functions into a single unit (class) while restricting direct access to the data. It helps protect data by allowing controlled access through member functions.

- Data members are typically hidden using the private access specifier.
- Public member functions (getters and setters) provide controlled access to the data.
- Improves data security, maintainability, and modularity.

Example: A Bank Account hides its balance using the private access specifier. The balance can only be accessed or modified through public functions such as deposit(), withdraw(), and getBalance().



Abstraction

Abstraction is the process of hiding implementation details and exposing only the essential features of an object. It allows users to interact with an object without knowing how its functionality is implemented internally.

- **Hides implementation details:** Users access only the required functionality while the internal logic remains hidden.
- **Improves maintainability:** Changes to the implementation do not affect the code using the abstraction.
- **Enhances flexibility:** Achieved using abstract classes and pure virtual functions.

Example: An ATM or a coffee machine demonstrates abstraction, where users perform operations such as withdrawing cash or selecting a beverage without knowing the internal implementation.



How to Achieve Abstraction in C++

Abstraction in C++ is achieved using abstract classes and pure virtual functions. An abstract class defines a common interface, while derived classes provide the implementation for the abstract functions.

Abstract Class: A class containing at least one pure virtual function. Objects of an abstract class cannot be created.

Pure Virtual Function: A virtual function declared using = 0 that must be overridden by derived classes.

Inheritance

Inheritance is a fundamental OOP concept in C++ that allows a class to acquire the data members and member functions of another class. It establishes an "is-a" relationship between classes and promotes code reuse.

- The class being inherited is called the base class, and the inheriting class is called the derived class.
- A derived class can access the members of the base class (subject to access specifiers) and can also define its own members.
- Promotes code reusability, extensibility, and hierarchical relationships**.**

Types of Inheritance

C++ supports the following types of inheritance:

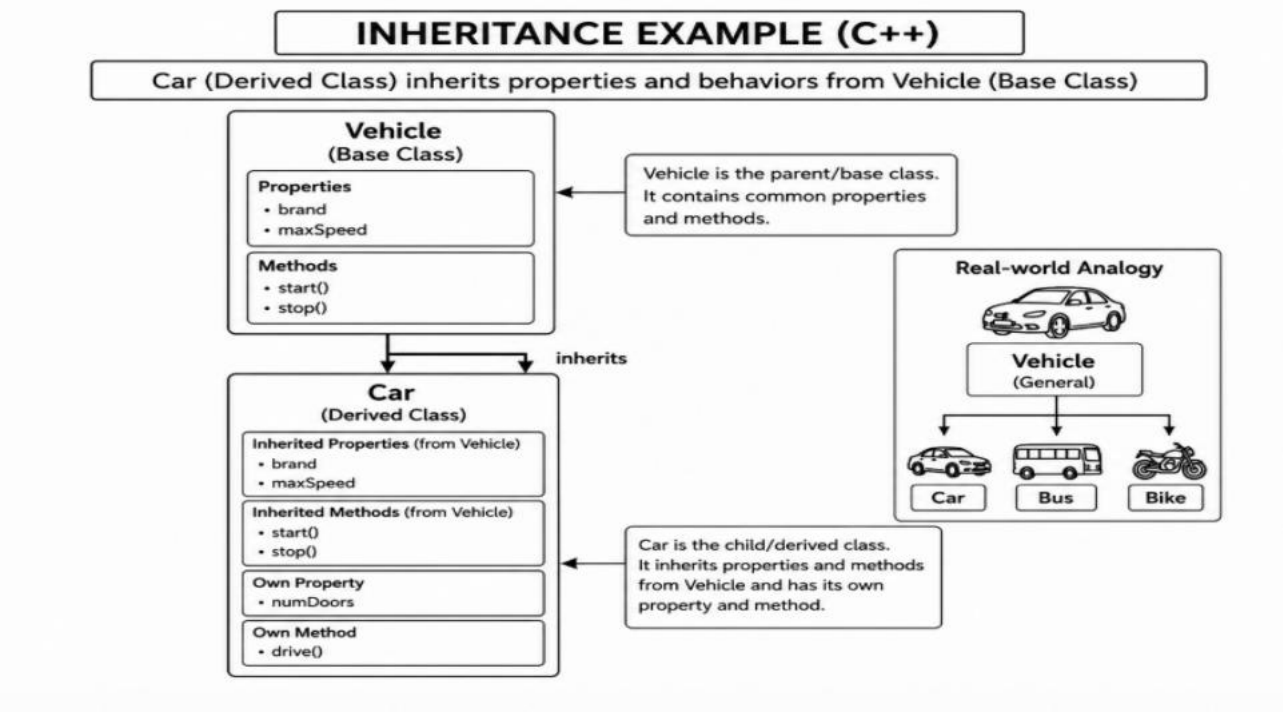
Single Inheritance: A derived class inherits from one base class.

Multiple Inheritance: A derived class inherits from more than one base class.

Multilevel Inheritance: A class is derived from another derived class, forming a chain of inheritance.

Hierarchical Inheritance: Multiple derived classes inherit from a single base class.

Hybrid Inheritance: A combination of two or more types of inheritance, such as multiple and multilevel inheritance.



Polymorphism

Polymorphism means "many forms", where the same interface can perform different actions depending on the object or context. In C++, it allows the same function or operator to exhibit different behaviour in different situations.

- Allows the same interface to perform different actions.
- Achieved through function overloading, operator overloading (compile-time polymorphism), and function overriding using virtual functions (run-time polymorphism).
- Improves code flexibility, reusability, and extensibility.

Advantages of OOP in C++

Object-Oriented Programming (OOP) offers several advantages over procedural programming by organizing programs around objects and classes.

- **Code Reusability**: Inheritance and classes enable existing code to be reused, reducing duplication.
- **Better Organization:** Programs are divided into logical classes, making the code easier to understand and maintain.
- **Improved Modularity:** Each class represents a separate unit, simplifying development, testing, and debugging.
- **Supports the DRY Principle:** Common functionality can be written once and reused across multiple classes.
- **Scalability:** Modular design makes it easier to extend and modify large applications.
- **Data Security:** Encapsulation restricts direct access to data and allows controlled access through member functions.

Limitations of OOP in C++

Although OOP provides many benefits, it also has some limitations.

Steeper Learning Curve: Concepts such as classes, inheritance, polymorphism, abstraction, and encapsulation may be difficult for beginners.

Higher Design Complexity: Designing effective class hierarchies and object relationships requires careful planning.

Additional Overhead: Small programs may require extra classes and objects, making the code more complex than necessary.

Higher Memory Usage: Objects and runtime features such as virtual functions can increase memory consumption.

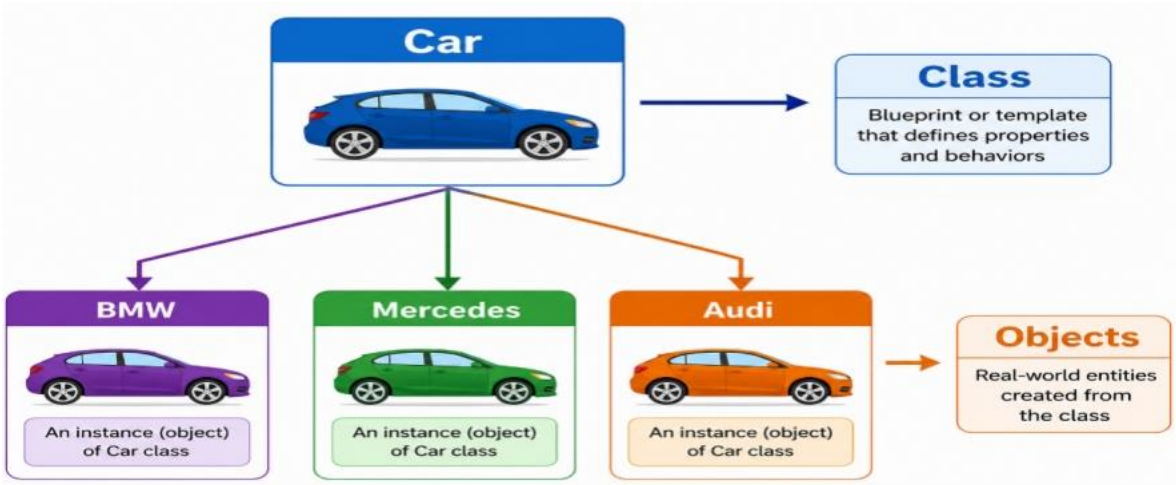
Performance Overhead: Features like dynamic binding and virtual functions may introduce a small runtime overhead compared to procedural code.

Classes and Objects

Class

A class is a user-defined data type holding data members and member functions. Data members are data variables, and member functions are used to manipulate these variables. Both data members and member functions define the properties and behavior of objects in a class.

Example: A Car represents a class (blueprint), while BMW, Mercedes, and Audi are objects (instances) created from that class.



C++ Class Syntax

```
class ClassName {
    access_specifier:
    // Body of the class
};
```

Object

An object in C++ is an instance of a class that represents a real-world entity. It is used to access the data members and member functions defined in the class.

- **State:** Represents the current values of an object's data members.
- **Behavior:** Represents the actions an object can perform through its member functions.
- **Identity:** Every object has a unique memory location that distinguishes it from other objects, even if they contain the same data.

Program

```
#include <iostream>
using namespace std;
// Class
class Car
{
public:
   string brand;
   void display()
   {
      cout << "Car Brand: " << brand << endl;
   }
};
int main()
{
   // Objects created from the Car class
   Car BMW;
   Car Mercedes;
   Car Audi;
   // Assign values
   BMW.brand = "BMW";
   Mercedes.brand = "Mercedes";
   Audi.brand = "Audi";
   // Display details
   BMW.display();
   Mercedes.display();
   Audi.display();
  return 0;
}
```

Output

Car Brand: BMW
Car Brand: Mercedes
Car Brand: Audi

Access Specifiers

In C++ access specifiers are keywords that decide how easily class elements, like data members and member functions can be reached. They manage which sections of a class are open, to parts of the program defining the degree of encapsulation and visibility.
C++ provides three types of access specifiers:

- Public
- Private
- Protected

Public-Public members can be reached from any part of the program outside the class and its subclasses.
Private-Private members are accessible only within the class they are declared in. They cannot be accessed by derived classes or other parts of the program, ensuring data hiding and encapsulation.
Protected-Protected members are like members but can also be accessed by classes that inherit from them. They are kept hidden from the rest of the program. Can be utilized in inheritance.
When a class is inherited the access specifier determines how the derived class interacts, with the base class members.

Example of Access Specifiers

```
#include<iostream>
using namespace std;
class MyClass {
public:
   int publicVar;      // Public member
protected:
   int protectedVar;   // Protected member
private:
   int privateVar;     // Private member
};
int main() {
   MyClass obj;
   obj.publicVar = 10; // Accessible and modifiable
   cout<<"Public Variable:"<<obj.publicVar<<endl;
   // obj.protectedVar = 20; --> Inaccessible and not modifiable directly
   // obj.privateVar = 30; --> Inaccessible and not modifiable
   return 0;
}
```

Output

```
Public Variable :10
```

Access specifiers are mainly used to achieve data hiding and encapsulation in C++.

Comparison

Specifier	Same Class	Derived Class	Outside Class
Private	✓	✗	✗
protected	✓	✓	✗
Public	✓	✓	✓

Constructors

Constructors are special methods that are automatically called whenever an object of a class is created. A constructor is different from normal functions in following ways:

- A constructor has same name as the class itself
- Don't have return type
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).

Consider a Car class. Every car may have properties such as **brand, model, color, and speed**. When a Car object is created, these properties need to be assigned suitable initial values. A constructor provides a convenient and systematic way to perform this initialization.

Syntax

```
class ClassName
{
public:
   ClassName()
   {
      // Initialization statements
   }};
```

Program

```
#include <iostream>
using namespace std;
class Car
{
public:
   string brand;
   string model;
   string color;
   int speed;
    // Constructor
   Car(string b, string m, string c, int s)
   {
      brand = b;
```

```
        model = m;
        color = c;
        speed = s;
    }
    void display()
    {
        cout << "Brand : " << brand << endl;
        cout << "Model : " << model << endl;
        cout << "Color : " << color << endl;
        cout << "Speed : " << speed << " km/h" << endl;
    }
};
int main()
{
    // Creating an object and initializing it using constructor
    Car car1("BMW", "X5", "Black", 200);
    car1.display();
    return 0;
```

Output

```
Brand : BMW
Model : X5
Color : Black
Speed : 200 km/h
```

Types of Constructors

Common constructor forms include:

1. Default constructor
2. Parameterized constructor
3. Copy constructor

There are 3 types of Constructors in C++. Let's learn about them.

1. **Default Constructor**

A constructor with no arguments (or parameters) in the definition is a default constructor. It is the type of constructor in C++ usually used to initialize data members (variables) with real values.

Note: The compiler automatically creates a default constructor without data member (variables) or initialization if no constructor is explicitly declared.

A constructor that takes no arguments.

Example

```
#include <iostream>
using namespace std;
class Student
{
```

```
public:
   Student()
   {
      cout << "Student created";
   }
};
int main()
{
   Student s;   // Object created
   return 0;
}
```

2.Parameterized Constructor

Unlike the Default constructor, It contains parameters (or arguments) in the constructor definition and declaration. More than one argument can also pass through a parameterized constructor.

A constructor that accepts arguments.

Program

```
#include <iostream>
using namespace std;
class Student
{
private:
   int rollNo;
public:
   // Parameterized constructor
   Student(int r)
   {
      rollNo = r;
   }
   void display()
   {
      cout << "Roll Number: " << rollNo;
   }
};
int main()
{
   Student s(101);   // Object creation

   s.display();
   return 0;
}
```

Output

101

Copy Constructor

A *copy constructor* initializes an object by copying the member values from an object of the same type. If your class members are all simple types such as scalar values, the compiler-generated copy constructor is sufficient and you don't need to define your own. If your class requires more complex initialization, then you need to implement a custom copy constructor.

For example, if a class member is a pointer then you need to define a copy constructor to allocate new memory and copy the values from the other's pointed-to object. The compiler-generated copy constructor simply copies the pointer, so that the new pointer still points to the other's memory location.

Syntax

```
ClassName(const ClassName &object)
{
   // copy data
}
```

Example

```
#include <iostream>
using namespace std;
class Student
{
private:
   int rollNo;
public:
   // Parameterized constructor
   Student(int r)
   {
      rollNo = r;
   }
   // Copy constructor
   Student(const Student &s)
   {
      rollNo = s.rollNo;
   }
   void display()
   {
      cout << "Roll Number: " << rollNo << endl;
   }
};
int main()
{
   Student s1(101);   // Original object
   Student s2(s1);    // Copy constructor
   s1.display();
```

```
    s2.display();
    return 0;
}
```

Output

```
Roll Number :101
Roll Number :101
```

Destructors

A destructor is a special member function, prefixed with '~', that is automatically called when an object goes out of scope or is destroyed to free resources like memory, files, or connections.

- Destructor neither requires any argument nor returns any value.
- A destructor cannot be declared as static. It also cannot be const-qualified because a destructor is meant to destroy the object, which inherently involves modifying its state.
- A class can have only one destructor, and it cannot be overloaded.
- Destructors release resources and destroy objects in reverse order of creation

Syntax

```
class ClassName
{
public:
    ~ClassName()
    {
        // Cleanup statements
    }
};
```

Example

```
#include <iostream>
using namespace std;
class Student
{
public:
    Student()
    {
        cout << "Student created" << endl;
    }
    ~Student()
    {
        cout << "Student destroyed" << endl;
    }
};
int main()
```

```
{
   Student s;
   cout << "Inside main function" << endl;
   return 0;
}
```

Output

```
Student created
Inside main function
Student destroyed
```

When s goes out of scope, the destructor is automatically called.

Constructor vs Destructor

Constructor	Destructor
Same name as class	Class name preceded by ~
Creates/initializes object	Cleans up object
Called during object creation	Called during object destruction
Can be overloaded	Cannot be overloaded
Can have parameters	Cannot have parameters

Constructor Overloading

Overloaded constructors have the same name (name of the class) but the different number of arguments. Depending upon the number and type of arguments passed, the corresponding constructor is called.

- Constructor overloading is an example of **compile-time polymorphism**.
- Multiple constructors can exist in the same class.
- Each constructor must have a **different parameter list**.
- The compiler selects the appropriate constructor based on the arguments.
- It provides different ways to initialize objects.
- Constructors cannot be overloaded only by changing the return type because constructors have no return type.

Syntax

```
class ClassName
{
public:
   ClassName()
   {
      // Default constructor
   }
   ClassName(int x)
   {
```

```
        // Constructor with one parameter
    }
    ClassName(int x, int y)
    {
        // Constructor with two parameters
    }
};
```

Example

```
#include <iostream>
using namespace std;
class Student
{
private:
   int rollNo;
   string name;
public:
   // Constructor 1: No arguments
   Student()
   {
      rollNo = 0;
      name = "Unknown";
   }
   // Constructor 2: One argument
   Student(int r)
   {
      rollNo = r;
      name = "Unknown";
   }
   // Constructor 3: Two arguments
   Student(int r, string n)
   {
      rollNo = r;
      name = n;
   }
   void display()
   {
      cout << "Roll Number: " << rollNo << endl;
      cout << "Name: " << name << endl;
      cout << endl;
   }
};
int main()
```

```
{
   Student s1;
   Student s2(101);
   Student s3(102, "Rahul");
   s1.display();
   s2.display();
   s3.display();
   return 0;
}
```

Output

```
Roll Number: 0
Name: Unknown
Roll Number: 101
Name: Unknown
Roll Number: 102
Name: Rahul
```

this Pointer

In C++, the this pointer is a special pointer available inside non-static member functions that points to the object invoking the function. It helps member functions identify and work with the current object on which they are called.

- Access and modify the data members and member functions of the current object.
- Resolve naming conflicts when function parameters and class data members have the same name.
- Return the current object from a member function to enable method chaining and fluent interfaces.

Example: The following example demonstrates how this differentiates between a data member and a constructor parameter with the same name.

Program

```
#include <iostream>
using namespace std;
class Student {
   int age;
public:
   Student(int age) {
      this->age = age;  // data member = parameter
   }
   void show() {
      cout << "Age: " << age << endl;
   }
};
int main()
```

```
{
   Student s(20);
   s.show();
   return 0;
}
```

Output

Age: 20

Static Members

Static data member

A **static data member** is declared inside a class using the static keyword, but unlike regular members, it **belongs to the class itself rather than individual objects**. This means there is **only one copy** of the variable, shared across all instances of the class, and its **lifetime spans the entire program execution** . Access control (public, private, protected) applies just like other class member.

Declaration and Definition

Static data members must be **declared inside the class** and **defined outside the class** using the scope resolution operator : :

Syntax

```
class ClassName
{
static data_type data_member;
};
data_type ClassName:: data_member=0;// Definition and initialization
```

Example

```
#include <iostream>
using namespace std;
class Employee {
public:
   static int Count;
   Employee() {
      Count++;
   }
};
// Definition and initialization of static member
int Employee::Count = 0;
int main() {
   Employee e1;
   Employee e2;
   Employee e3;
```

```
    cout << "Number of Employee objects created: "
         << Employee::Count << endl;
    return 0;
}
```

Output:

Number of Employee objects created: 3

Accessing Static Members

Static data members can be accessed in two ways:

***Using the class name and scope resolution operator**

```
Employee::Count;
```

***Using an object of the class**

```
Employee emp;
emp.Count;
```

Static members can be accessed even if no objects of the class exist

Characteristics of Static Data Members:

- Only one copy of a static data member exists for the entire class, shared by all objects.
- It is initialized once and exists independently of class objects.
- Its lifetime spans the entire program execution.
- Access control (public, private, protected) applies just like other class members.

Static member function

A static member function in C++ is a function that belongs to the class rather than any specific object of the class. It is declared using the static keyword inside the class and can be called without creating an object.

- Can be invoked using the class name without creating an object.
- Can access only static data members and other static member functions.
- Shared among all objects of the class.
- Commonly used for utility methods and managing class-level data.

Program

```
#include <iostream>
using namespace std;
class Box
{
   private:
   static int length;
   static int breadth;
   static int height;
   public:
   static void print()
   {
      cout << "The value of the length is: " << length << endl;
      cout << "The value of the breadth is: " << breadth << endl;
```

```
        cout << "The value of the height is: " << height << endl;
    }
};
// initialize the static data members
int Box :: length = 25;
int Box :: breadth = 20;
int Box :: height = 60;
int main()
{
    cout << "\nStatic member function is called through Class name" << endl;
    Box::print();
    return 0;
}
```

Output

```
Static member function is called through Class name
The value of the length is: 25
The value of the breadth is: 20
The value of the height is: 60
```

Static members can be used for **method-call tracking** in:

- **Debugging** – find how frequently a method is executed.
- **Performance monitoring** – identify frequently called methods.
- **Logging** – maintain the number of operations performed.
- **Statistics** – count how many times a particular operation occurs.
- **Testing** – verify whether a method was called the expected number of times.

const member functions

The const member functions are the functions which are declared as constant in the program. The object called by these functions cannot be modified. It is recommended to use const keyword so that accidental changes to object are avoided.

A const member function can be called by any type of object. Non-const functions can be called by non-const objects only.

syntax

```
datatype function_name const();
```

Example

```
#include <iostream>
using namespace std;
class Circle {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
```

```
    double getArea() const {
        return 3.14159 * radius * radius;
    }
};
int main() {
    const Circle myCircle(5.0);
    cout << "Area: " << myCircle.getArea() << endl;

return 0;
}
```

Output

Area: 78.53975

Friend Function

A friend function is a non-member function that has permission to access the private and protected members of a class. It is declared inside the class using the keyword friend.

Characteristics of Friend Functions

*A friend function is declared inside a class but defined outside of it.
*It is not a member of the class, so it cannot be called using the class object directly.
*It can access private and protected members of the class it is declared as a friend of.

Syntax

```
class ClassName
{
friend  returnType  functionName(parameters);
};
```

Example

```
#include <iostream>
using namespace std;
class Student
{
private:
    int marks;
public:
    Student()
    {
        marks = 90;
    }
    friend void display(Student s); // Friend function declaration
};
```

```
void display(Student s) // Friend function definition
{
   cout << "Student Marks: " << s.marks << endl;
}
int main()
{
   Student s;
   display(s);
   return 0;
}
```

Output

Student Marks: 90

A friend class

A friend class can access private and protected members of other classes in which it is declared as a friend. This is useful when two classes need to work closely together and share internal data.

Characteristics of Friend Class

- Declared using the friend keyword inside the class.
- Friendship is **not mutual** — if A is a friend of B, B is not automatically a friend of A.
- Friendship is **not inherited** — derived classes do not automatically become friends.
- Should be used **sparingly** to maintain encapsulation.

Program

```
#include <iostream>
using namespace std;
class Student
{
private:
   int marks = 75;
   friend class Teacher; // Friend class declaration
};
class Teacher
{
public:
   void display(Student s)
   {
      // Accessing private member of Student
      cout << "Student Marks: " << s.marks << endl;
   }
};
int main()
{
   Student s;
```

```
    Teacher t;
    t.display(s);
    return 0;
}
```

Output

Student Marks: 75

Scope resolution operator

Scope resolution operator :: is used to specify the scope in which an identifier (variable, function, class, etc.) is defined. It helps the compiler distinguish between variables or functions with the same name in different scopes.

Main Uses of :: in C++

Accessing Global Variables

It is especially useful when a **local variable and global variable have the same name**.

When a local variable hides a global variable, :: can be used to access the global one.

Program

```
#include <iostream>
using namespace std;
int x = 10; // Global variable
int main() {
    int x = 20; // Local variable
    cout << "Local x: " << x << endl;
    cout << "Global x: " << ::x << endl; // Access global variable
    return 0;
}
```

Output

Local x: 20

Global x: 10

Defining Class Member Functions Outside the Class

In C++, a class member function can be **declared inside the class and defined outside the class**. When the function is defined outside the class, the **scope resolution operator (::)** is used to specify that the function belongs to a particular class. This approach separates the class declaration from the actual implementation of its functions, making the program more organized, readable, and easier to maintain. The function is first declared inside the class, and its definition is written outside using the

syntax

return_type ClassName::functionName().

The scope resolution operator tells the compiler that the function is a member of the specified class. This method is commonly used in larger programs where class declarations are placed in header files and function definitions are placed separately in source files.

Program

```
#include <iostream>
using namespace std;
class Student
{
private:
   int marks;

public:
   void setMarks(int m);
   void display();
};
void Student::setMarks(int m)
{
   marks = m;
}
void Student::display()
{
   cout << "Marks = " << marks << endl;
}
int main()
{
   Student s;
   s.setMarks(85);
   s.display();
   return 0;
}
```

Output

Marks = 85